

NORTH CAROLINA STATE UNIVERSITY
TEXTILE ENGINEERING

TE 205 – Textile Engineering Systems
Experiment # 9 – Controlling the Pi

1 Objectives

1. Determine how to incorporate LEDs into circuits
2. Learn how to interface the Raspberry Pi with a circuit
3. Practice writing and running Python programs for the Raspberry Pi

2 Homework

1. Finish the programming assignments up to assignment 8
2. Determine the minimum valued resistor for use with an LED
3. Skim the lab before answering the questions.
4. Fill out the questions in the lab BEFORE it begins.

3 Problem

A Raspberry Pi has a wealth of uses and can be a powerful tool for developing new technologies and devices; however, the learning curve is quite large and deters many entrepreneurs and innovators from utilizing them. The purpose of this lab is to familiarize you with the Raspberry Pi and particularly how to control output with it.

Individual goals include:

1. Developing a circuit and software to control a single LED
2. Developing a circuit and software to control a tricolor LED

4 Procedure

HARDWARE:

LEDs

LEDs are a pretty fantastic device for many of the reasons you already know: such as they're highly efficient, they come in a myriad of wonderful colors, and they're pretty easy to use in a circuit once you get past this one minor detail. Their resistance is more difficult to understand than that one Russian book about war and peace. Ironically there's no sparknotes. Let's see why it's so strange with a few graphs on the next page.

The graph on the right is the voltage versus current for a normal ohmic resistor. As you can see, the current rises relatively constantly. If you sought to make an equation to relate the two, it would look like this:

$$V = IR$$

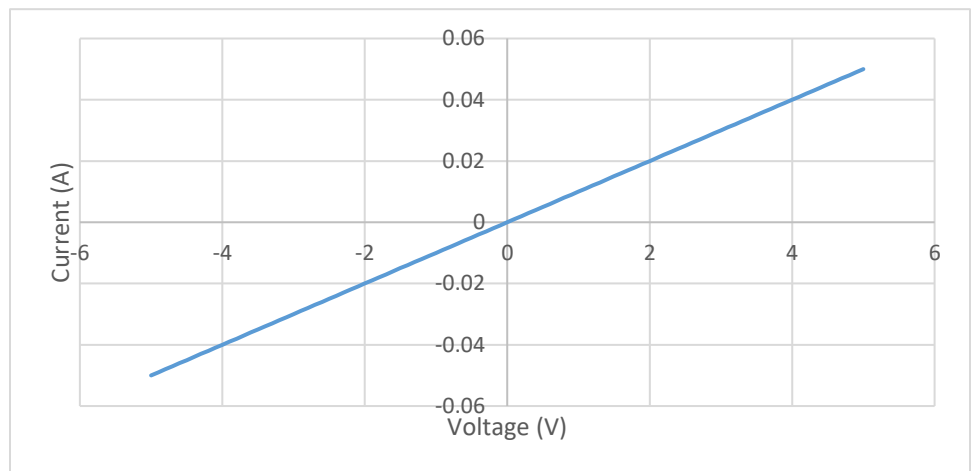


Figure 1- Voltage vs. Current for an Ohmic Resistor

That's an equation that you're familiar with. You can even determine the resistance of the resistor from the slope of the line.

Question 1. Estimate the resistance of the resistor in Figure 1: _____ Ω

In figure 2 you can see the voltage vs. resistance for a typical LED. The resistance certainly isn't constant and that makes it more difficult to work with. Instead there are two to three numbers worth keeping track for most LEDs:

1. The forward voltage is how much voltage is required to normally run the LED
2. The maximum safe current that can be run through the LED is a very important number to keep track of.
3. The breakdown (reverse) voltage is required to "break" an LED if it is run backwards. It's the voltage at which electricity can run through it in reverse, exceeding it is unhealthy for the LED.

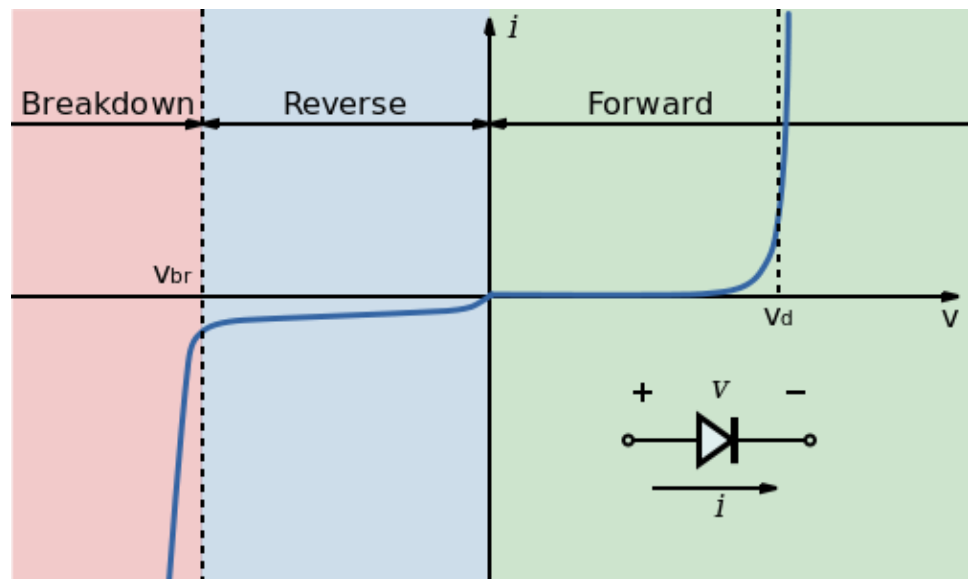


Figure 2- Current vs voltage for typical LED. All credit goes to wikipedia for this image

Here is a link to the datasheet for an LED very similar to the one we're using in lab:

<https://cdn-shop.adafruit.com/datasheets/WP7113SRD-D.pdf> (if you google LED datasheet then it's the adafruit one for a "super bright red" LED)

Question 2. Determine the following values from the above datasheet:

Typical Forward Voltage:	Max Current:	Max Reverse Voltage:
V	mA	V

As mentioned earlier, an LED has a direction to it. A diode is a device that only lets current pass through in one direction (although from figure 2 we can see it doesn't do this perfectly).

In figure 3 do you see how the arrow points to the right? That's because diodes point in the direction that electricity should flow through them. Then the other part of the icon, the arrows coming off of it are to indicate it's a LIGHT EMITTING diode.

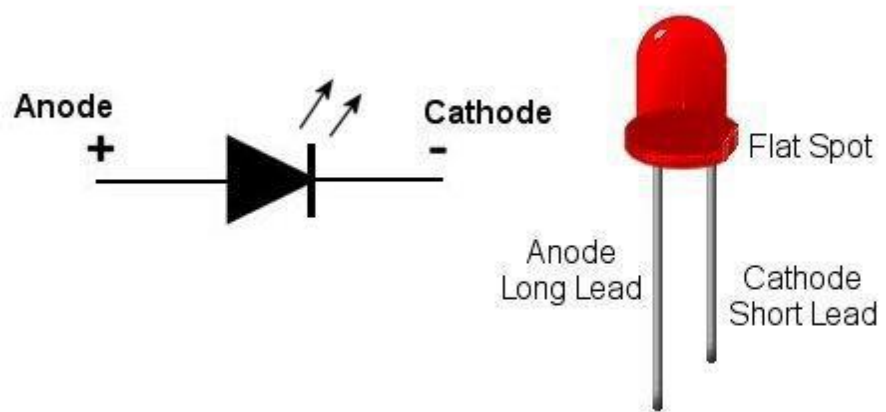
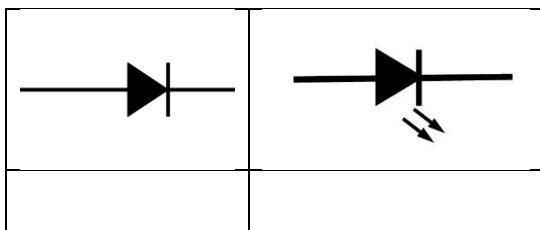


Figure 3 - Light Emitting Diode (LED)

Question 3. Label the each of the following figures as a diode or LED:



Diodes are directional, and now you know how to read their circuit diagrams, but let's talk about the physical LED. There are two legs on them, like resistors; however, the two legs are intentionally different lengths to make distinguishing their direction easier. The long leg is where the current enters, and the short leg is where it goes. Here are a couple of ways to remember this: the electrons fall from high

to low energy to release light so they go from the long to the short leg. The electrons are excited when they go in and are tired when they leave, and so they can't go as far.

Question 4. Create your own method for recalling that the long lead goes with the higher voltage for LEDs:

So the final question is how do we safely incorporate our LED into our circuit? The two main limiting factors (assuming we don't plug it in backwards) are the typical forward voltage and the maximum allowed current. That means that we can treat it as an element with a known voltage and current and solve for the rest of the circuit from there.

Question 5. Using the space provided to the right, create a circuit with a resistor and a 3.3 V voltage source that can safely operate our LED (assume $V = 2\text{ V}$ and $I_{\text{max}} = 20\text{ mA}$)
Resistor value = _____ Ω

Question 5.

The GPIO Header

The 40-pin GPIO header is located along the side of the board opposite the HDMI/power connectors. GPIO stands for **General Purpose Input Output**, which in this case means digital I/O. It is a little inconvenient to connect test wires directly to the pins, although they do make male/female leads where one side connects to a GPIO pin and the other to the breadboard. So one of the first things you will do is to connect the 40-pin ribbon cable to GPIO header.



Be very careful when you place the ribbon cable on the header. All the pins must line up exactly before applying any pressure or you will bend or damage the pins on the board.

Specific pins can be configured to be an input pin or an output pin. If a pin is set as an input pin, then one reads the state of the pin, which is externally set to be either high or low. In the case of the Raspberry Pi, high (true) means 3.3V and low (false) mean 0V. Even though the Raspberry Pi has a 5V power supply (pins 2 and 4), it is a 3.3V device and 3.3V is the maximum voltage you can safely connect to the GPIO input pins.



Never connect a wire directly between the 3.3V (or 5V) power-output pin and the GPIO input or you will damage the board. It is important when you are building and testing your circuit to never do this or you will permanently damage the board. In addition, **never connect a pin directly to ground in the output state**, as each pin can only source 16mA of current, and the Raspberry Pi can only supply a maximum of 50 mA to all the pins.

Question 6. Based on the information above, what is the new maximum current that can be driven through the LED in our circuit? Rewrite your previous circuit using the space below to take this into account:

New Resistor value: _____ Ω

Question 6.

The GPIO Ribbon allows you to plug the GPIO header directly into a breadboard. This makes it slightly easier to access your Raspberry Pi's GPIO pins. Table 1 describes how the pins on the raspberry Pi are linked to the ribbon, and how they might line up on the breadboard. The pins in table 1 come in many different colors. The grey pins are grounds. The green pins are GPIO pins that can be used as output or input. The gold pins are 3.3V voltage sources and the red pins are 5 V voltage sources. The last two are ... special. Don't worry about them. Also, don't use them for any of the work we're

Table 1: pinouts and signal names of the J8 40 pin connector

Function/GPIO	J8 Pin		Function/GPIO
3.3V	1	2	5.0V
GPIO2	3	4	5.0V
GPIO3	5	6	0V
GPIO4	7	8	GPIO14
0V	9	10	GPIO15
GPIO17	11	12	GPIO18
GPIO27	13	14	0V
GPIO22	15	16	GPIO23
3.3V	17	18	GPIO24
GPIO10	19	20	0V
GPIO9	21	22	GPIO25
GPIO11	23	24	GPIO8
0V	25	26	GPIO7
(GPIO0) ID_SD	27	28	ID_SC (GPIO1)
GPIO5	29	30	0V
GPIO6	31	32	GPIO12
GPIO13	33	34	0V
GPIO19	35	36	GPIO16
GPIO26	37	38	GPIO20
0V	39	40	GPIO21

performing in this lab.

Using the Raspberry Pi as a power source

Use pins 1 (power source) and 6 (ground) to create the circuit that you detailed in question 6 (not question 5). The light should turn on! If it doesn't turn on, then you should certainly check whether the Raspberry Pi is on, whether the correct ports have been utilized, and whether the LED is in the correct direction.

Connecting to the Raspberry Pi

In Windows, select Start->Accessories->Remote Desktop Connection. You should see an icon like the one shown in Figure 3. Type in the name of the Raspberry Pi computer at the dialog box. It should be `tex-raspberry#.tx.ncsu.edu` where # is a number between 4 and 16. Once connected, you will see a login dialog box as shown in Figure 4.

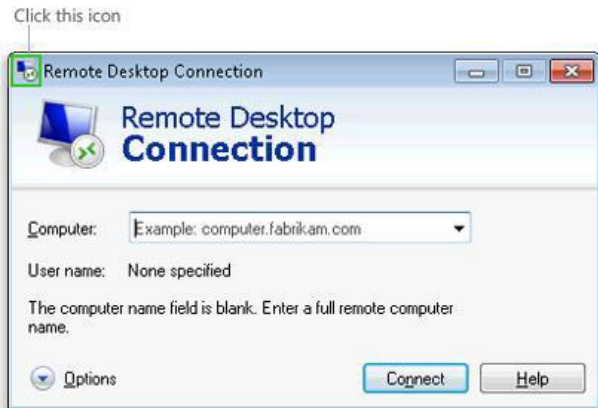


Figure 4- remote desktop connection

The user name is `pi` and the password is `textiles pi` in all lower case. There is a single space between the word *textiles* and *pi*. Then left click the OK button.

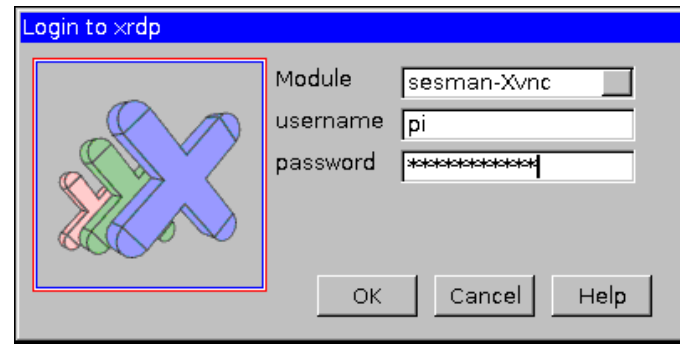


Figure 5- login screen to Raspberry Pi

Configuring the GPIO pins from python

From Python, you can control the GPIO pins using the `RPi.GPIO` module. You will adapt python assignment 4 and 8 to run on Raspberry Pis instead in this lab. For this we need lots of knowledge, particularly about how GPIO works for the Raspberry Pi. For this, let's write the example in the Python on the Raspberry Pi so it will be particularly fresh in our minds. The non-pi version of the code is on the left, and the pi version is on the right. Please write the Pi version of the code instead of the Desktop version.

Desktop version (No Raspberry Pi):

```
import time
a = False

while True:
    print(a)
    time.sleep(1)
    a = not a
```

Raspberry Pi version:

```
import RPi.GPIO
import time
RPi.GPIO.setmode(RPi.GPIO.BCM)
RPi.GPIO.setup(3, RPi.GPIO.OUT)
a = False

while True:
    RPi.GPIO.output(3, a)
    time.sleep(1)
    a = not a
```

Desktop version (No Raspberry Pi):

```
import time
a = False

while True:
    print(a)
    time.sleep(1)
    a = not a
```

Raspberry Pi version:

```
import RPi.GPIO
import time
RPi.GPIO.setmode(RPi.GPIO.BCM)
RPi.GPIO.setup(3, RPi.GPIO.OUT)
a = False

while True:
    RPi.GPIO.output(3, a)
    time.sleep(1)
    a = not a
```

Question 7. Circle the differences between the two versions of the code above.

The first difference between the two versions is `RPi.GPIO`. Because we're importing it, you should be able to recognize that it's a library. In particular it's the class `GPIO` in the library `RPi` that contains a wealth of utility for the Raspberry Pi. To find more information about it, look up `RPi.GPIO` and you'll be able to look up information about the current system (you may need to go to the sourceforge link to find the actual documentation).

Question(s) 8. Is RPIO suitable for timing critical operations? _____

Is PWM supported on the RPIO? _____

What website hosts documentation for the RPi.GPIO? _____

The next difference is the `setmode` method. The pins on a raspberry pi can be referenced in two different ways:

1. `RPi.GPIO.BOARD` states that when you pass the value 3 into functions in the `RPi.GPIO` module, you are referring to pin 3 (which happens to be `GPIO2`).
2. `RPi.GPIO.BCM` states that when you pass the value 3 into the functions in the `RPi.GPIO` module you are referring to the signal `GPIO3` (which happens to be pin 5).

After `setmode` comes the `setup` method. The `setup` method is used to set a pin to be used as input or output. One of the options for this method is obviously `RPi.GPIO.OUT`, the other two can be found in the `RPi` documentation under `examples/input`.

Each of the above methods only need to be called once when setting up the Raspberry Pi. The next method, `RPi.GPIO.output(3,True*)`, turns on `GPIO3`, (or pin 5). As you could imagine, `RPi.GPIO.output(3,False*)` does exactly the opposite.

Question 9. Look up the PWM function for the Rpi.GPIO and familiarize yourself with it for the next section.

Converting code for the Raspberry Pi

At this point you know the individual tools required for this lab and all that's required is to combine them with the code that you already have and circuits to make your two basic devices. You already have the basic circuit for the first program, which controls a single LED. I'll convert my version of assignment 3 to RPi code to show you how to do this.

Assignment Version

```
import time
import virtual
# The virtual library comes with the class PWM which may
# called as it is below. The PWM has several defined functions:
# it's constructor PWM(pin, freq) requires the pin and
# of the PWM mode. Keep the duty cycle between 0-100.
# .start(duty) starts the pwm cycle at the duty cycle specified
# .ChangeDutyCycle(duty) changes the duty cycle
# .stop() turns the PWM off.
# The duty and 'on-ness' of the PWM can be checked with .duty
and .on

#for example, this line creates a PWM class called x
x = virtual.PWM(3, 50)

#this line turns this PWM on with a duty cycle of 50
x.start(50)
#prompt allows my input line below to fit in this column
prompt = "Please enter a command for the PWM system:\n"

command = "red means go"
while command != "exit":
    command = input(prompt)
    if command == "pulse":
        while True:
            for i in range(x.duty, 101, 1):
                x.ChangeDutyCycle(i)
                time.sleep(.02)
            for i in range(x.duty, -1, -1):
                x.ChangeDutyCycle(i)
                time.sleep(.02)
```

Raspberry Pi Version

```
import time
import RPi.GPIO as g

g.setmode(RPi.GPIO.BCM)
g.setup(3, RPi.GPIO.OUT)

x = g.PWM(3,50)

dc = 50
x.start(dc)

command = "red means go"
while command != "exit":
    command = input("enter command\n")
    if command == "pulse":
        while True:
            for i in range(dc, 101, 1):
                dc = i
                x.ChangeDutyCycle(dc)
                time.sleep(0.02)
            for i in range(dc, -1, -1):
                x.ChangeDutyCycle(dc)
                time.sleep(0.02)
```

As you can see there are quite a few changes that needed to be made to our code to change it from the python code that we've been using to code that's compatible with the Raspberry Pi. In particular, the variable `dc` was introduced to keep track of the duty cycle of the pin being used. The PWM class in `Rpi.GPIO` doesn't support direct referencing of the duty cycle of the pin so we need our own variable to keep track of it (although there are other clever methods as well). We also added the `setmode` and `setup` commands at the beginning. Note that because the keyword "`as`" was used after `import RPi.GPIO`, the unwieldy name `RPi.GPIO` could be simplified to `g`.

Modify the circuit you designed in part one to run assignment four. The tricolor LED module required to build the circuit in for assignment eight will be introduced in the lab.